

Analise_MI_MAGIC_pitch.R

Ricardo Ruiz

2026-07-13

```
#####
# PROVA DE CONCEITO COM DADOS REAIS:
# INFORMACAO MUTUA EM UMA POPULACAO MAGIC DE FEIJAO
#
# Objetivos:
# 1) Ler os dados fenotipicos e genotipicos do Harvard Dataverse;
# 2) Calcular Informacao Mutua (MI) entre cada SNP e diferentes caracteres;
# 3) Verificar a concentracao dos SNPs mais informativos em regioes de QTL
#    descritas por Diaz et al. (2020);
# 4) Comparar GBLUP e MIBLUP em validacao cruzada para produtividade em 2014;
# 5) Gerar figuras e tabelas para o pitch.
#
# Arquivos esperados na mesma pasta:
# 01. MAGIC_raw_data.csv
# 03. MAGIC_model_data.csv
# 05. MAGIC_Pvulgaris_GBS.vcf.gz
# 06. MAGIC_genetic_map.csv
#
# Observacao:
# Os BLUES e BLUPs presentes em 03. MAGIC_model_data.csv ja foram obtidos pelos
# autores apos o ajuste espacial dos ensaios. O codigo usa BLUP por padrao.
#####

rm(list = ls())
options(stringsAsFactors = FALSE)
set.seed(20260713)

#####
# 0) CONFIGURACOES
#####

# Altere apenas esta pasta.
base_dir <- "."

raw_file <- file.path(base_dir, "01. MAGIC_raw_data.csv")
model_file <- file.path(base_dir, "03. MAGIC_model_data.csv")
vcf_file <- file.path(base_dir, "05. MAGIC_Pvulgaris_GBS.vcf.gz")
map_file <- file.path(base_dir, "06. MAGIC_genetic_map.csv")

out_dir <- file.path(base_dir, "resultados_MI_MAGIC")
fig_dir <- file.path(out_dir, "figuras")
```

```

dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE)

# Parametros principais
METHOD_PHENO <- "BLUP"
N_BINS       <- 10
MAX_MISSING  <- 0.20
MIN_MAF      <- 0.05
TOP_PROP     <- 0.01

# A validacao cruzada e a parte mais demorada.
RUN_CV       <- TRUE
CV_FOLDS     <- 5
CV_REPEATS   <- 3
CV_SEED      <- 20260713

#####
# 1) PACOTES
#####

required_pkgs <- c(
  "vcfR",
  "data.table",
  "dplyr",
  "tidyr",
  "tibble",
  "ggplot2",
  "patchwork",
  "rrBLUP"
)

install_if_missing <- function(pkgs) {
  missing <- pkgs[!vapply(pkgs, requireNamespace, logical(1), quietly = TRUE)]
  if (length(missing) > 0) {
    install.packages(missing, dependencies = TRUE)
  }
}

install_if_missing(required_pkgs)

library(vcfR)

##
##      *****      ***   vcfR   ***      *****
##      This is vcfR 1.16.0
##      browseVignettes('vcfR') # Documentation
##      citation('vcfR') # Citation
##      *****      *****      *****      *****

library(data.table)
library(dplyr)

##

```

```

## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:data.table':
##
##   between, first, last

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(tidyr)
library(tibble)
library(ggplot2)
library(patchwork)
library(rrBLUP)

#####
# 2) LEITURA DOS DADOS FENOTIPICOS
#####

stopifnot(
  file.exists(raw_file),
  file.exists(model_file),
  file.exists(vcf_file),
  file.exists(map_file)
)

raw_data  <- data.table::fread(raw_file, data.table = FALSE)
model_data <- data.table::fread(model_file, data.table = FALSE)
gen_map   <- data.table::fread(map_file, data.table = FALSE)

# O arquivo original possui espacos ao final de SdFe e SdZn.
names(raw_data)  <- trimws(names(raw_data))
names(model_data) <- trimws(names(model_data))
names(gen_map)   <- trimws(names(gen_map))

cat("\nDimensoes dos arquivos fenotipicos:\n")

##
## Dimensoes dos arquivos fenotipicos:

cat("Dados brutos:", nrow(raw_data), "linhas x", ncol(raw_data), "colunas\n")

## Dados brutos: 6868 linhas x 20 colunas

cat("Dados ajustados:", nrow(model_data), "linhas x", ncol(model_data), "colunas\n")

## Dados ajustados: 2502 linhas x 10 colunas

```

```
#####
# 3) LEITURA E CONVERSAO DO VCF
#####

# O cache evita reler e converter o VCF em todas as execucoes.
cache_file <- file.path(out_dir, "MAGIC_genotipos_cache.rds")

if (file.exists(cache_file)) {

  cat("\nLendo matriz genotipica do cache...\n")
  cache <- readRDS(cache_file)
  geno <- cache$geno
  marker_info <- cache$marker_info

} else {

  cat("\nLendo o VCF. Esta etapa pode demorar alguns minutos...\n")
  vcf <- vcfR::read.vcfR(vcf_file, verbose = FALSE)

  fix <- as.data.frame(vcfR::getFIX(vcf), stringsAsFactors = FALSE)
  gt <- vcfR::extract.gt(vcf, element = "GT", as.numeric = FALSE)

  if (anyDuplicated(fix$ID)) {
    stop("O VCF possui identificadores de marcadores duplicados.")
  }

  # Conversao bialelica:
  # 0/0 -> 0; 0/1 ou 1/0 -> 1; 1/1 -> 2; ausente -> NA.
  dosage_by_variant <- matrix(
    NA_real_,
    nrow = nrow(gt),
    ncol = ncol(gt),
    dimnames = dimnames(gt)
  )

  dosage_by_variant[gt %in% c("0/0", "0|0")] <- 0
  dosage_by_variant[gt %in% c("0/1", "1/0", "0|1", "1|0")] <- 1
  dosage_by_variant[gt %in% c("1/1", "1|1")] <- 2

  # Indivíduos nas linhas e marcadores nas colunas.
  geno <- t(dosage_by_variant)
  rownames(geno) <- colnames(gt)
  colnames(geno) <- fix$ID

  marker_info <- data.frame(
    Marker = fix$ID,
    Chromosome = fix$CHROM,
    Position = as.numeric(fix$POS),
    stringsAsFactors = FALSE
  )

  saveRDS(
    list(geno = geno, marker_info = marker_info),

```

```

    cache_file,
    compress = TRUE
  )

  rm(vcf, fix, gt, dosage_by_variant)
  invisible(gc())
}

```

```

##
## Lendo matriz genotipica do cache...

```

```

cat("Matriz genotipica:", nrow(geno), "individuos x",
    ncol(geno), "marcadores\n")

```

```

## Matriz genotipica: 637 individuos x 20810 marcadores

```

```

#####
# 4) FUNCOES AUXILIARES
#####

#-----
# 4.1 Controle de qualidade dos marcadores
#-----

qc_markers <- function(X, max_missing = 0.20, min_maf = 0.05) {

  missing_rate <- colMeans(is.na(X))

  allele_freq <- colMeans(X, na.rm = TRUE) / 2
  maf <- pmin(allele_freq, 1 - allele_freq)

  keep <- (
    missing_rate <= max_missing &
    is.finite(maf) &
    maf >= min_maf
  )

  list(
    keep = keep,
    table = data.frame(
      Marker = colnames(X),
      MissingRate = missing_rate,
      MAF = maf,
      Keep = keep
    )
  )
}

#-----
# 4.2 Moda de cada SNP e imputacao
#-----

```

```

marker_modes <- function(X) {

  counts <- rbind(
    colSums(X == 0, na.rm = TRUE),
    colSums(X == 1, na.rm = TRUE),
    colSums(X == 2, na.rm = TRUE)
  )

  # max.col recebe uma matriz com marcadores nas linhas.
  max.col(t(counts), ties.method = "first") - 1
}

fill_missing_with_modes <- function(X, modes) {

  cols_missing <- which(colSums(is.na(X)) > 0)

  for (j in cols_missing) {
    X[is.na(X[, j]), j] <- modes[j]
  }

  X
}

impute_mode <- function(X) {
  modes <- marker_modes(X)
  list(
    X = fill_missing_with_modes(X, modes),
    modes = modes
  )
}

impute_train_test <- function(X_train, X_test) {

  modes <- marker_modes(X_train)

  list(
    train = fill_missing_with_modes(X_train, modes),
    test = fill_missing_with_modes(X_test, modes),
    modes = modes
  )
}

#-----
# 4.3 Informacao Mutua discreta, calculada de forma vetorizada
#
# O fenotipo e discretizado em classes de mesma amplitude por:
# cut(y, breaks = n_bins, include.lowest = TRUE)
#
# A MI e expressa em bits porque se utiliza log2.
#-----

entropy_columns <- function(P) {
  P_log <- P

```

```

P_log[P_log <= 0] <- 1
-colSums(P * log2(P_log))
}

compute_mi_fast <- function(X, y, n_bins = 10) {

  stopifnot(
    nrow(X) == length(y),
    !anyNA(X),
    !anyNA(y)
  )

  y_disc <- cut(
    y,
    breaks = n_bins,
    labels = FALSE,
    include.lowest = TRUE
  )

  y_factor <- factor(y_disc)
  Y <- model.matrix(~ y_factor - 1)

  n <- nrow(X)
  m <- ncol(X)

  py <- colMeans(Y)
  Hy <- -sum(py[py > 0] * log2(py[py > 0]))

  p_genotype <- rbind(
    colMeans(X == 0),
    colMeans(X == 1),
    colMeans(X == 2)
  )

  Hx <- entropy_columns(p_genotype)

  Hxy <- numeric(m)

  for (g in 0:2) {
    joint_counts <- crossprod(Y, X == g)
    joint_prob <- joint_counts / n
    Hxy <- Hxy + entropy_columns(joint_prob)
  }

  mi <- Hx + Hy - Hxy
  pmax(mi, 0)
}

#-----
# 4.4 Preparar uma combinacao ano-caracteristica
#-----

prepare_trait_data <- function(

```

```

year,
trait,
method = "BLUP",
genotype_matrix = geno,
phenotype_data = model_data,
marker_data = marker_info,
max_missing = MAX_MISSING,
min_maf = MIN_MAF,
n_bins = N_BINS,
top_prop = TOP_PROP) {

if (!trait %in% names(phenotype_data)) {
  stop("Característica não encontrada: ", trait)
}

pheno <- phenotype_data |>
  filter(
    .data$Year == year,
    .data$Method == method,
    !is.na(.data[[trait]]),
    .data$Line %in% rownames(genotype_matrix)
  ) |>
  select(Line, all_of(trait))

if (anyDuplicated(pheno$Line)) {
  stop("Existem linhas repetidas para ", trait, " em ", year, ".")
}

X_raw <- genotype_matrix[pheno$Line, , drop = FALSE]
y <- as.numeric(pheno[[trait]])

qc <- qc_markers(
  X_raw,
  max_missing = max_missing,
  min_maf = min_maf
)

X_qc <- X_raw[, qc$keep, drop = FALSE]
imp <- impute_mode(X_qc)
X <- imp$X

mi <- compute_mi_fast(X, y, n_bins = n_bins)

idx_info <- match(colnames(X), marker_data$Marker)

if (anyNA(idx_info)) {
  stop("Ha marcadores sem informacao de posicao no VCF.")
}

mi_table <- marker_data[idx_info, , drop = FALSE] |>
  mutate(MI = mi) |>
  arrange(desc(MI), Marker) |>
  mutate(
    Rank = row_number(),

```

```

    Top = Rank <= ceiling(n() * top_prop)
  )

list(
  year = year,
  trait = trait,
  method = method,
  lines = pheno$Line,
  y = y,
  X = X,
  qc = qc$table,
  mi_table = mi_table,
  n_individuals = nrow(X),
  n_markers = ncol(X)
)
}

#-----
# 4.5 Resumo da sobreposicao com uma regio conhecida
#-----

summarize_qtl_window <- function(
  analysis,
  qtl_chr,
  qtl_start,
  qtl_end,
  label,
  top_prop = TOP_PROP,
  reference_marker = NA_character_) {

  tab <- analysis$mi_table
  top_n <- ceiling(nrow(tab) * top_prop)

  in_window <- (
    tab$Chromosome == qtl_chr &
    tab$Position >= qtl_start &
    tab$Position <= qtl_end
  )

  top_in_window <- sum(in_window[seq_len(top_n)])
  markers_in_window <- sum(in_window)

  expected_hits <- top_n * markers_in_window / nrow(tab)

  fold_enrichment <- if (expected_hits > 0) {
    top_in_window / expected_hits
  } else {
    NA_real_
  }

  reference_rank <- NA_integer_
  reference_mi <- NA_real_

```

```

if (!is.na(reference_marker)) {
  reference_rank <- match(reference_marker, tab$Marker)
  if (!is.na(reference_rank)) {
    reference_mi <- tab$MI[reference_rank]
  }
}

tibble(
  Analysis = label,
  Year = analysis$year,
  Trait = analysis$trait,
  Method = analysis$method,
  Individuals = analysis$n_individuals,
  Markers = analysis$n_markers,
  TopN = top_n,
  QTLChromosome = qtl_chr,
  QTLStart = qtl_start,
  QTLEnd = qtl_end,
  MarkersInWindow = markers_in_window,
  TopMarkersInWindow = top_in_window,
  ExpectedByChance = expected_hits,
  FoldEnrichment = fold_enrichment,
  ReferenceMarker = reference_marker,
  ReferenceMarkerRank = reference_rank,
  ReferenceMarkerMI = reference_mi
)
}

#-----
# 4.6 Posicoes acumuladas para grafico tipo Manhattan
#-----

add_cumulative_position <- function(tab) {

  plot_data <- tab |>
    mutate(
      ChrNumber = as.integer(gsub("[^0-9]", "", Chromosome))
    ) |>
    arrange(ChrNumber, Position)

  chr_info <- plot_data |>
    group_by(ChrNumber) |>
    summarise(ChrLength = max(Position), .groups = "drop") |>
    arrange(ChrNumber) |>
    mutate(
      Offset = c(0, head(cumsum(ChrLength), -1)),
      Center = Offset + ChrLength / 2
    )

  plot_data <- plot_data |>
    left_join(chr_info, by = "ChrNumber") |>
    mutate(
      CumulativePosition = Position + Offset,

```

```

    ChrGroup = factor(ChrNumber %% 2)
  )

  list(data = plot_data, chr_info = chr_info)
}

#-----
# 4.7 Grafico de MI ao longo do genoma
#-----

plot_mi_manhattan <- function(
  analysis,
  qtl_chr,
  qtl_start,
  qtl_end,
  title,
  top_prop = TOP_PROP) {

  pos <- add_cumulative_position(analysis$mi_table)
  d <- pos$data
  chr_info <- pos$chr_info

  qtl_chr_number <- as.integer(gsub("[^0-9]", "", qtl_chr))
  qtl_offset <- chr_info$Offset[chr_info$ChrNumber == qtl_chr_number]

  rect_start <- qtl_offset + qtl_start
  rect_end <- qtl_offset + qtl_end

  top_n <- ceiling(nrow(d) * top_prop)
  mi_cut <- sort(d$MI, decreasing = TRUE)[top_n]

  top_marker <- d |>
    slice_max(MI, n = 1, with_ties = FALSE)

  ggplot(d, aes(x = CumulativePosition, y = MI)) +
    annotate(
      "rect",
      xmin = rect_start,
      xmax = rect_end,
      ymin = -Inf,
      ymax = Inf,
      alpha = 0.16
    ) +
    geom_point(
      aes(color = ChrGroup),
      size = 0.65,
      alpha = 0.72
    ) +
    geom_hline(
      yintercept = mi_cut,
      linetype = "dashed",
      linewidth = 0.45
    ) +

```

```

    annotate(
      "text",
      x = top_marker$CumulativePosition,
      y = top_marker$MI,
      label = top_marker$Marker,
      hjust = -0.03,
      vjust = -0.35,
      size = 2.6
    ) +
    scale_color_manual(
      values = c("0" = "grey35", "1" = "grey68"),
      guide = "none"
    ) +
    scale_x_continuous(
      breaks = chr_info$Center,
      labels = paste0("Pv", sprintf("%02d", chr_info$ChrNumber)),
      expand = expansion(mult = c(0.01, 0.08))
    ) +
    labs(
      title = title,
      subtitle = paste0(
        "Faixa sombreada: regio de QTL conhecida; linha tracejada: top ",
        100 * top_prop, "% dos SNPs"
      ),
      x = "Cromossomo",
      y = "Informacao Mutua (bits)"
    ) +
    theme_minimal(base_size = 10) +
    theme(
      panel.grid.major.x = element_blank(),
      panel.grid.minor = element_blank(),
      axis.text.x = element_text(size = 8),
      plot.title = element_text(face = "bold"),
      plot.margin = margin(8, 35, 8, 8)
    ) +
    coord_cartesian(clip = "off")
}

#####
# 5) MAPA DO ENSAIO DE CAMPO
#####

raw_2014 <- raw_data |>
  filter(
    .data$Year == 2014,
    !is.na(.data$Yd),
    !is.na(.data[["Field row"]]),
    !is.na(.data[["Field column"]])
  )

p_field <- ggplot(
  raw_2014,
  aes(

```

```

    x = .data[["Field column"]],
    y = .data[["Field row"]],
    fill = .data$Yd
  )
) +
geom_tile(width = 0.95, height = 0.95) +
scale_y_reverse() +
scale_fill_viridis_c(
  option = "C",
  name = expression("Produtividade (kg ha"^{-1}*")")
) +
coord_fixed() +
labs(
  title = "Distribuicao espacial da produtividade no ensaio de 2014",
  subtitle = "Delineamento alfa-lattice com tres repeticoes e coordenadas de linha e coluna",
  x = "Coluna no campo",
  y = "Linha no campo"
) +
theme_minimal(base_size = 11) +
theme(
  panel.grid = element_blank(),
  plot.title = element_text(face = "bold")
)

ggsave(
  file.path(fig_dir, "01_mapa_campo_produtividade_2014.png"),
  p_field,
  width = 8.5,
  height = 8,
  dpi = 300
)

#####
# 6) ANALISES DE INFORMACAO MUTUA
#####

analysis_specs <- tribble(
  ~Key,      ~Label,      ~Year, ~Trait, ~QTLChr, ~QTLStart, ~QTLEnd, ~
  "Yd_2013",  "Produtividade (2013)",  2013,  "Yd",   "Chr01",  10000000,  18000000,
  "DPM_2014", "Maturidade fisiologica (2014)",  2014,  "DPM",   "Chr08",    0,    1600000,
  "SdFe_2014", "Ferro na semente (2014)",  2014,  "SdFe",  "Chr06",  21000000,  24000000,
  "PHI_2013", "Indice de colheita da vagem (2013)",  2013,  "PHI",   "Chr02",  47000000,  48000000,
)

mi_analyses <- list()
qtl_summaries <- list()
mi_plots <- list()

for (i in seq_len(nrow(analysis_specs))) {

  spec <- analysis_specs[i, ]

  cat(

```

```

    "\nCalculando MI para", spec$Label,
    "-", METHOD_PHENO, "... \n"
  )

  result <- prepare_trait_data(
    year = spec$Year,
    trait = spec$Trait,
    method = METHOD_PHENO
  )

  mi_analyses[[spec$Key]] <- result

  qtl_summaries[[spec$Key]] <- summarize_qtl_window(
    analysis = result,
    qtl_chr = spec$QTLChr,
    qtl_start = spec$QTLStart,
    qtl_end = spec$QTLEnd,
    label = spec$Label,
    reference_marker = spec$ReferenceMarker
  )

  mi_plots[[spec$Key]] <- plot_mi_manhattan(
    analysis = result,
    qtl_chr = spec$QTLChr,
    qtl_start = spec$QTLStart,
    qtl_end = spec$QTLEnd,
    title = spec$Label
  )

  data.table::fwrite(
    result$mi_table,
    file.path(out_dir, paste0("MI_", spec$Key, "_completo.csv"))
  )

  data.table::fwrite(
    head(result$mi_table, 100),
    file.path(out_dir, paste0("MI_", spec$Key, "_top100.csv"))
  )

  ggsave(
    file.path(fig_dir, paste0("MI_", spec$Key, "_Manhattan.png")),
    mi_plots[[spec$Key]],
    width = 11,
    height = 4.8,
    dpi = 300
  )
}

```

```

##
## Calculando MI para Produtividade (2013) - BLUP ...

```

```

##
## Calculando MI para Maturidade fisiologica (2014) - BLUP ...

```

```
##
## Calculando MI para Ferro na semente (2014) - BLUP ...

##
## Calculando MI para Indice de colheita da vagem (2013) - BLUP ...
```

```
qtl_summary <- bind_rows(qtl_summaries)

data.table::fwrite(
  qtl_summary,
  file.path(out_dir, "resumo_sobreposicao_QTL.csv")
)

cat("\nResumo reproduzível da sobreposicao com QTLs:\n")
```

```
##
## Resumo reproduzível da sobreposicao com QTLs:
```

```
print(
  qtl_summary |>
  select(
    Analysis,
    Individuals,
    Markers,
    TopN,
    MarkersInWindow,
    TopMarkersInWindow,
    ExpectedByChance,
    FoldEnrichment,
    ReferenceMarkerRank
  )
)
```

```
## # A tibble: 4 x 9
##   Analysis      Individuals Markers TopN MarkersInWindow TopMarkersInWindow
##   <chr>          <int>   <int> <dbl>         <int>             <int>
## 1 Produtividade (2~      444   12167  122           109              43
## 2 Maturidade fisio~      602   12203  123           313             123
## 3 Ferro na semente~      601   12221  123           268              80
## 4 Indice de colhei~      444   12167  122            54              12
## # i 3 more variables: ExpectedByChance <dbl>, FoldEnrichment <dbl>,
## #   ReferenceMarkerRank <int>
```

```
#####
# 7) FIGURA COM QUATRO CARACTERISTICAS
#####
```

```
p_mi_combined <-
  mi_plots[["Yd_2013"]] +
  mi_plots[["DPM_2014"]] +
  mi_plots[["SdFe_2014"]] +
  mi_plots[["PHI_2013"]] +
```

```

plot_layout(ncol = 2) +
plot_annotation(
  title = "Informacao Mutua recupera regioes genomicas conhecidas em dados reais",
  subtitle = paste0(
    "Fenotipos ajustados por ", METHOD_PHENO,
    "; ", N_BINS, " classes de mesma amplitude; ",
    "MAF >= ", MIN_MAF, " e ausencias <= ", 100 * MAX_MISSING, "%"
  )
)

ggsave(
  file.path(fig_dir, "02_MI_quatro_caracteristicas.png"),
  p_mi_combined,
  width = 15,
  height = 10,
  dpi = 300
)

#####
# 8) GRAFICO DE ENRIQUECIMENTO NAS REGIOES DE QTL
#####

p_enrichment <- qtl_summary |>
mutate(
  Analysis = reorder(Analysis, FoldEnrichment),
  Label = paste0(
    TopMarkersInWindow, " SNPs; ",
    sprintf("%.1f", FoldEnrichment), "x"
  )
) |>
ggplot(aes(x = Analysis, y = FoldEnrichment)) +
geom_col(width = 0.68) +
geom_hline(yintercept = 1, linetype = "dashed", linewidth = 0.5) +
geom_text(
  aes(label = Label),
  hjust = -0.08,
  size = 3.4
) +
coord_flip(clip = "off") +
scale_y_continuous(expand = expansion(mult = c(0, 0.20))) +
labs(
  title = "Enriquecimento dos SNPs mais informativos em regioes de QTL",
  subtitle = paste0(
    "Razao entre o numero observado no top ",
    100 * TOP_PROP,
    "% e o esperado pela proporcao de SNPs na regioao"
  ),
  x = NULL,
  y = "Fator de enriquecimento"
) +
theme_minimal(base_size = 11) +
theme(
  panel.grid.major.y = element_blank(),

```

```

    plot.title = element_text(face = "bold"),
    plot.margin = margin(8, 45, 8, 8)
  )

ggsave(
  file.path(fig_dir, "03_enriquecimento_regioes_QTL.png"),
  p_enrichment,
  width = 9,
  height = 5.2,
  dpi = 300
)

#####
# 9) VALIDACAO CRUZADA: GBLUP VS MIBLUP PARA PRODUTIVIDADE EM 2014
#####

safe_cor <- function(x, y) {
  if (sd(x) == 0 || sd(y) == 0) {
    return(NA_real_)
  }
  cor(x, y)
}

rmse <- function(obs, pred) {
  sqrt(mean((obs - pred)^2))
}

prepare_cv_data <- function(
  year = 2014,
  trait = "Yd",
  method = "BLUP",
  genotype_matrix = geno,
  phenotype_data = model_data,
  max_missing = MAX_MISSING,
  min_maf = MIN_MAF) {

  pheno <- phenotype_data |>
    filter(
      .data$Year == year,
      .data$Method == method,
      !is.na(.data[[trait]]),
      .data$Line %in% rownames(genotype_matrix)
    ) |>
    select(Line, all_of(trait))

  X <- genotype_matrix[pheno$Line, , drop = FALSE]
  y <- as.numeric(pheno[[trait]])

  qc <- qc_markers(
    X,
    max_missing = max_missing,
    min_maf = min_maf
  )
}

```

```

X <- X[, qc$keep, drop = FALSE]

list(
  X = X,
  y = y,
  lines = pheno$Line,
  qc = qc$table
)
}

fit_predict_rrblup <- function(Z_train, y_train, Z_test) {

  fit <- rrBLUP::mixed.solve(
    y = y_train,
    Z = Z_train,
    method = "REML"
  )

  as.numeric(fit$beta) + as.numeric(Z_test %*% fit$u)
}

run_cv_gblup_miblup <- function(
  X,
  y,
  folds = 5,
  repeats = 3,
  seed = 20260713,
  n_bins = 10) {

  n <- nrow(X)
  results <- list()
  counter <- 1

  for (r in seq_len(repeats)) {

    set.seed(seed + r)
    fold_id <- sample(rep(seq_len(folds), length.out = n))

    for (f in seq_len(folds)) {

      cat("CV: repeticao", r, "de", repeats,
        "- fold", f, "de", folds, "\n")

      idx_test <- which(fold_id == f)
      idx_train <- setdiff(seq_len(n), idx_test)

      X_train_raw <- X[idx_train, , drop = FALSE]
      X_test_raw <- X[idx_test, , drop = FALSE]

      y_train <- y[idx_train]
      y_test <- y[idx_test]

      # Imputacao determinada apenas pelo conjunto de treino.

```

```

imp <- impute_train_test(X_train_raw, X_test_raw)
X_train <- imp$train
X_test <- imp$test

# Remocao de SNPs sem variacao dentro do fold de treino.
train_mean <- colMeans(X_train)
train_var <- colMeans(X_train^2) - train_mean^2
keep_var <- is.finite(train_var) & train_var > 1e-12

X_train <- X_train[, keep_var, drop = FALSE]
X_test <- X_test[, keep_var, drop = FALSE]

# Centralizacao determinada somente no treino.
center_train <- colMeans(X_train)
Z_train <- sweep(X_train, 2, center_train, FUN = "-")
Z_test <- sweep(X_test, 2, center_train, FUN = "-")

# ----- GBLUP -----
t0 <- proc.time()[3]

pred_gblup <- fit_predict_rrblup(
  Z_train = Z_train,
  y_train = y_train,
  Z_test = Z_test
)

time_gblup <- proc.time()[3] - t0

# ----- MIBLUP -----
t0 <- proc.time()[3]

# A MI e calculada apenas com os individuos do treino.
mi <- compute_mi_fast(
  X = X_train,
  y = y_train,
  n_bins = n_bins
)

if (!is.finite(mean(mi)) || mean(mi) <= 1e-12) {
  weights <- rep(1, length(mi))
} else {
  weights <- mi / mean(mi)
}

weights <- pmax(weights, 1e-8)

Zw_train <- sweep(Z_train, 2, sqrt(weights), FUN = "*")
Zw_test <- sweep(Z_test, 2, sqrt(weights), FUN = "*")

pred_miblup <- fit_predict_rrblup(
  Z_train = Zw_train,
  y_train = y_train,
  Z_test = Zw_test
)

```

```

)

time_miblup <- proc.time()[3] - t0

results[[counter]] <- tibble(
  Repeat = r,
  Fold = f,
  Model = "GBLUP",
  NTrain = length(idx_train),
  NTest = length(idx_test),
  Markers = ncol(Z_train),
  Correlation = safe_cor(y_test, pred_gblup),
  RMSE = rmse(y_test, pred_gblup),
  TimeSeconds = time_gblup
)

counter <- counter + 1

results[[counter]] <- tibble(
  Repeat = r,
  Fold = f,
  Model = "MIBLUP",
  NTrain = length(idx_train),
  NTest = length(idx_test),
  Markers = ncol(Z_train),
  Correlation = safe_cor(y_test, pred_miblup),
  RMSE = rmse(y_test, pred_miblup),
  TimeSeconds = time_miblup
)

counter <- counter + 1
}
}

bind_rows(results)
}

if (RUN_CV) {

  cat("\nPreparando a validacao cruzada de produtividade em 2014...\n")

  cv_data <- prepare_cv_data(
    year = 2014,
    trait = "Yd",
    method = METHOD_PHENO
  )

  cat(
    "CV com", nrow(cv_data$X), "individuos e",
    ncol(cv_data$X), "marcadores apos o controle de qualidade.\n"
  )

  cv_results <- run_cv_gblup_miblup(

```

```

X = cv_data$X,
y = cv_data$y,
folds = CV_FOLDS,
repeats = CV_REPEATS,
seed = CV_SEED,
n_bins = N_BINS
)

cv_results$Model <- factor(
  cv_results$Model,
  levels = c("GBLUP", "MIBLUP")
)

cv_summary <- cv_results |>
  group_by(Model) |>
  summarise(
    MeanCorrelation = mean(Correlation, na.rm = TRUE),
    SDCorrelation = sd(Correlation, na.rm = TRUE),
    SECORrelation = SDCorrelation / sqrt(sum(is.finite(Correlation))),
    LowerCorrelation = MeanCorrelation -
      qt(0.975, df = sum(is.finite(Correlation)) - 1) * SECORrelation,
    UpperCorrelation = MeanCorrelation +
      qt(0.975, df = sum(is.finite(Correlation)) - 1) * SECORrelation,
    MeanRMSE = mean(RMSE, na.rm = TRUE),
    SDRMSE = sd(RMSE, na.rm = TRUE),
    MeanTimeSeconds = mean(TimeSeconds, na.rm = TRUE),
    .groups = "drop"
  )

cv_paired <- cv_results |>
  select(Repeat, Fold, Model, Correlation, RMSE) |>
  pivot_wider(
    names_from = Model,
    values_from = c(Correlation, RMSE)
  ) |>
  mutate(
    DeltaCorrelation = Correlation_MIBLUP - Correlation_GBLUP,
    DeltaRMSE = RMSE_GBLUP - RMSE_MIBLUP
  )

data.table::fwrite(
  cv_results,
  file.path(out_dir, "CV_GBLUP_MIBLUP_Yd_2014_resultados.csv")
)

data.table::fwrite(
  cv_summary,
  file.path(out_dir, "CV_GBLUP_MIBLUP_Yd_2014_resumo.csv")
)

data.table::fwrite(
  cv_paired,
  file.path(out_dir, "CV_GBLUP_MIBLUP_Yd_2014_ganhos_pareados.csv")
)

```

```

)

cat("\nResumo da validacao cruzada:\n")
print(cv_summary)

cat("\nGanhos medios pareados do MIBLUP sobre o GBLUP:\n")
print(
  cv_paired |>
    summarise(
      MeanDeltaCorrelation = mean(DeltaCorrelation, na.rm = TRUE),
      ProportionBetterCorrelation = mean(
        DeltaCorrelation > 0,
        na.rm = TRUE
      ),
      MeanDeltaRMSE = mean(DeltaRMSE, na.rm = TRUE),
      ProportionBetterRMSE = mean(
        DeltaRMSE > 0,
        na.rm = TRUE
      )
    )
)

p_cv_cor <- ggplot(
  cv_results,
  aes(
    x = Model,
    y = Correlation,
    group = interaction(Repeat, Fold)
  )
) +
  geom_line(alpha = 0.30, linewidth = 0.45) +
  geom_point(size = 2.0) +
  stat_summary(
    aes(group = Model),
    fun = mean,
    geom = "point",
    shape = 18,
    size = 4
  ) +
  labs(
    title = "Acuracia preditiva",
    subtitle = "Cada linha liga os resultados do mesmo fold",
    x = NULL,
    y = "Correlacao entre observado e predito"
  ) +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"))

p_cv_rmse <- ggplot(
  cv_results,
  aes(
    x = Model,
    y = RMSE,

```

```

    group = interaction(Repeat, Fold)
  )
) +
geom_line(alpha = 0.30, linewidth = 0.45) +
geom_point(size = 2.0) +
stat_summary(
  aes(group = Model),
  fun = mean,
  geom = "point",
  shape = 18,
  size = 4
) +
labs(
  title = "Erro de predicacao",
  subtitle = "Valores menores de RMSE indicam melhor desempenho",
  x = NULL,
  y = "RMSE"
) +
theme_minimal(base_size = 11) +
theme(plot.title = element_text(face = "bold"))

p_cv <- p_cv_cor + p_cv_rmse +
plot_annotation(
  title = "Validacao cruzada em dados reais: GBLUP versus MIBLUP",
  subtitle = paste0(
    CV_REPEATS, " repeticoes de ", CV_FOLDS,
    " folds para produtividade em 2014; pesos de MI calculados apenas no treino"
  )
)

ggsave(
  file.path(fig_dir, "04_CV_GBLUP_MIBLUP_Yd_2014.png"),
  p_cv,
  width = 11,
  height = 5.5,
  dpi = 300
)
}

```

```

##
## Preparando a validacao cruzada de produtividade em 2014...
## CV com 602 individuos e 12203 marcadores apos o controle de qualidade.
## CV: repeticao 1 de 3 - fold 1 de 5
## CV: repeticao 1 de 3 - fold 2 de 5
## CV: repeticao 1 de 3 - fold 3 de 5
## CV: repeticao 1 de 3 - fold 4 de 5
## CV: repeticao 1 de 3 - fold 5 de 5
## CV: repeticao 2 de 3 - fold 1 de 5
## CV: repeticao 2 de 3 - fold 2 de 5
## CV: repeticao 2 de 3 - fold 3 de 5
## CV: repeticao 2 de 3 - fold 4 de 5
## CV: repeticao 2 de 3 - fold 5 de 5
## CV: repeticao 3 de 3 - fold 1 de 5

```

```
## CV: repeticao 3 de 3 - fold 2 de 5
## CV: repeticao 3 de 3 - fold 3 de 5
## CV: repeticao 3 de 3 - fold 4 de 5
## CV: repeticao 3 de 3 - fold 5 de 5
##
## Resumo da validacao cruzada:
## # A tibble: 2 x 9
##   Model MeanCorrelation SDCorrelation SECORrelation LowerCorrelation
##   <fct>          <dbl>          <dbl>          <dbl>          <dbl>
## 1 GBLUP          0.245          0.0985          0.0254          0.191
## 2 MIBLUP          0.251          0.100          0.0259          0.195
## # i 4 more variables: UpperCorrelation <dbl>, MeanRMSE <dbl>, SDRMSE <dbl>,
## #   MeanTimeSeconds <dbl>
##
## Ganhos medios pareados do MIBLUP sobre o GBLUP:
## # A tibble: 1 x 4
##   MeanDeltaCorrelation ProportionBetterCorr~1 MeanDeltaRMSE ProportionBetterRMSE
##   <dbl>          <dbl>          <dbl>          <dbl>
## 1          0.00536          0.667          0.105          0.667
## # i abbreviated name: 1: ProportionBetterCorrelation
```

```
#####
# 10) INFORMACOES DA EXECUCAO
#####
```

```
writeLines(
  capture.output(sessionInfo()),
  file.path(out_dir, "sessionInfo.txt")
)

cat("\nAnalise concluida.\n")
```

```
##
## Analise concluida.
```

```
cat("Tabelas:", normalizePath(out_dir, winslash = "/", mustWork = FALSE), "\n")
```

```
## Tabelas: C:/Users/Ricardo Ruiz/Desktop/Doutorado Estatistica Exp Agronomica/Leitura - GDM/MAGIC Beans
```

```
cat("Figuras:", normalizePath(fig_dir, winslash = "/", mustWork = FALSE), "\n")
```

```
## Figuras: C:/Users/Ricardo Ruiz/Desktop/Doutorado Estatistica Exp Agronomica/Leitura - GDM/MAGIC Beans
```